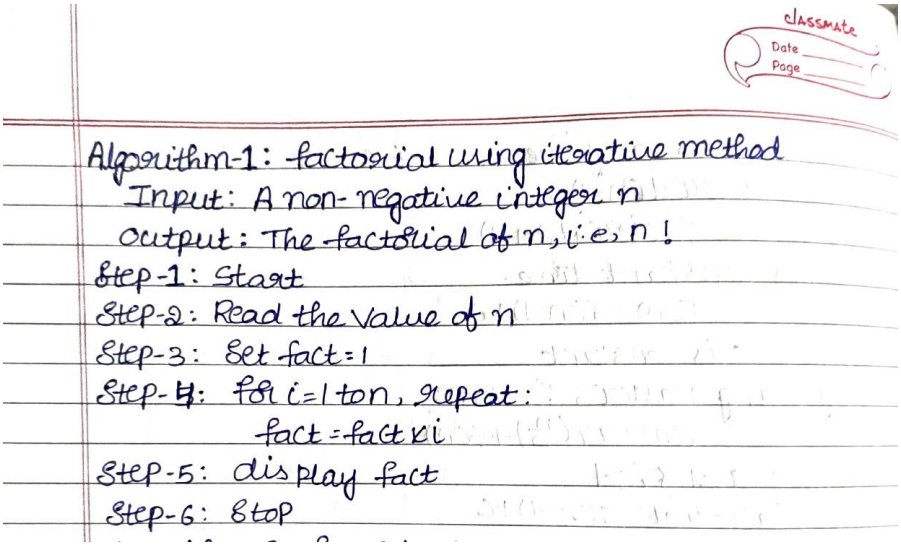
PRACTICAL-4

Aim: To Implementation and Time analysis of factorial program using iterative and recursive method

Algorithm:



Algorithm-1: factorial using iterative method
Input: A non-negative integer n
output: The factorial of n, i.e, n!
Step-1: Start
Step-2: Read the value of n
Step-3: Set fact=1
Step-4: for i=1 to n, repeat:
fact=fact*i
Step-5: display fact
Step-6: Stop

Code:

```
//iterative Method
 #include <iostream>

using namespace std;

 class Factorial

{

public:

int fact(int n)

   {

      int result = 1;

       for (int i = 1; i <= n; i++)

      {

         result = result * i;

      }

 return result;

   }

};
```

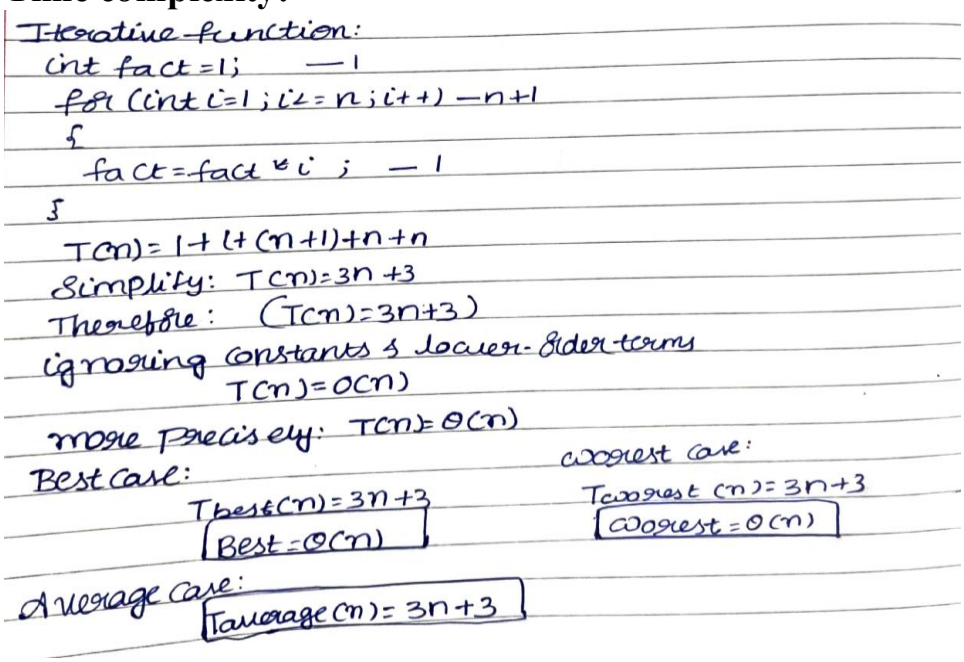
```
int main()
{
 int n,result;
 Factorial obj;
cout << "Enter the value to find its factorial: ";
 cin >> n;
result = obj.fact(n);
cout << "Factorial of " << n << " is " << result << endl;
 return 0;
}
```

Output:

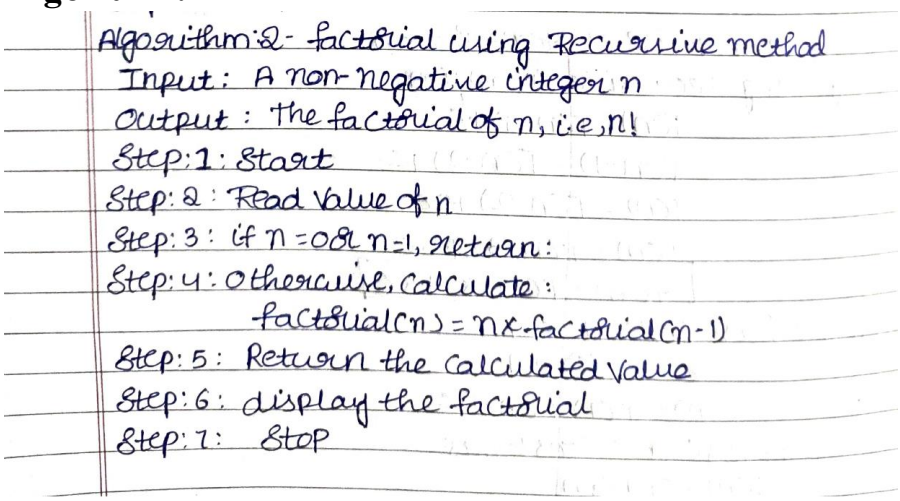
```
    + FullyQualifiedErrorId : CommandNotFoundException

PS C:\Desktop\dda-practicals> C:\Desktop\dda-practicals/practical-4.exe
Enter the value to find its factorial: 6
Factorial of 6 is 720
PS C:\Desktop\dda-practicals>
```

Time complexity:



Algorithm:



Code:

```
//Recursive method
 #include <iostream>

using namespace std;

 class Factorial

{

public:

   int factr(int n)

   {

      if (n == 0 || n == 1)

      {

         return 1;

      }

      else

      {

         int result = n * factr(n - 1);

         return result;

      }

   }

};

 int main()

{

   int n, result;
```

```
    // Create an object of the class
    Factorial obj;
    cout << "Enter the value to find its factorial: ";
    cin >> n;
     result = obj.factr(n);
    cout << "Factorial of " << n << " is " << result << endl;
    return 0;
}
```

Output:

```
PS C:\Desktop\dda-practicals> C:\Desktop\dda-practicals/practical-4.2.exe
Enter the value to find its factorial: 7
Factorial of 7 is 5040
PS C:\Desktop\dda-practicals>
```

Time complexity:

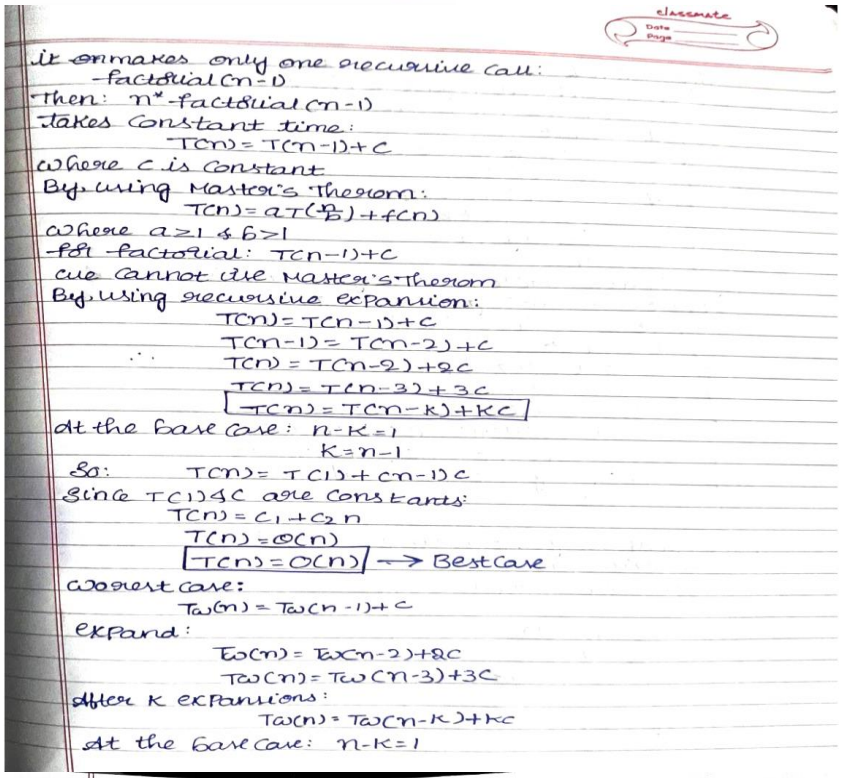
Time Complexity: Recursive method

1. Recursive factorial:

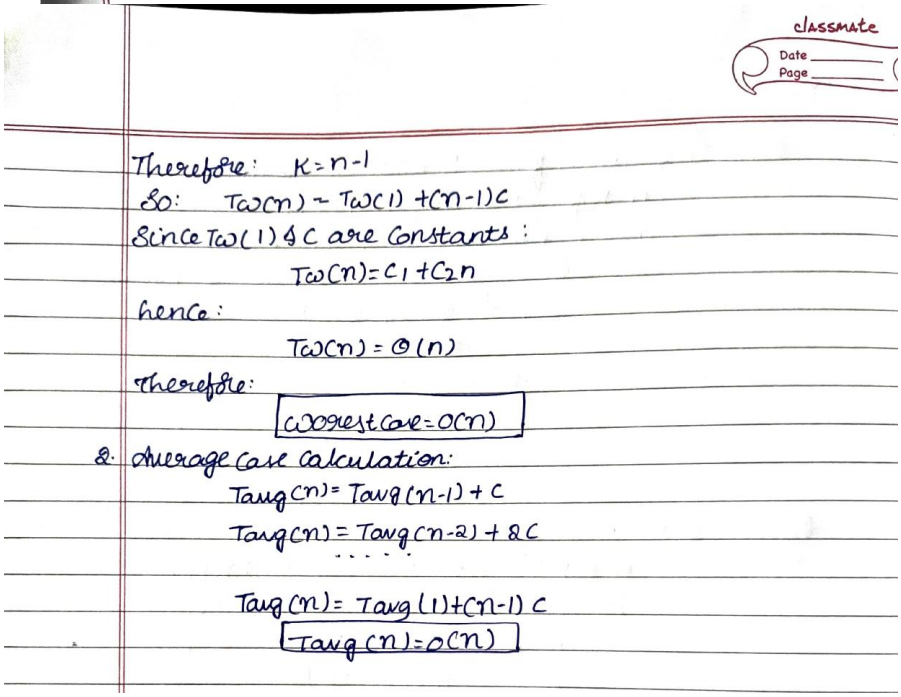
```
int factorial(int n)
{
  if(n==0||n==1)
      return 1;
  return n*factorial(n-1);
}
```

for every call:

factorial(n)



it only makes only one recursive call:
-factorial(n-1)
Then: n*-factorial(n-1)
takes constant time:
T(n) = T(n-1)+c
where c is constant
By using Master's Theorem:
T(n) = aT(n/b)+f(n)
where a≥1 & b>1
for factorial: T(n-1)+c
we cannot use Master's Theorem
By using recursive expansion:
T(n) = T(n-1)+c
T(n-1) = T(n-2)+c
T(n) = T(n-2)+2c
T(n) = T(n-3)+3c
T(n) = T(n-k)+kc
At the base case: n-k=1
k=n-1
So: T(n) = T(1)+(n-1)c
Since T(1) & c are constants:
T(n) = c1+c2 n
T(n) = O(n)
T(n) = O(n) → Best Case
worst case:
Tw(n) = Tw(n-1)+c
expand:
Tw(n) = Tw(n-2)+2c
Tw(n) = Tw(n-3)+3c
After k expansions:
Tw(n) = Tw(n-k)+kc
At the base case: n-k=1



Therefore: k=n-1
So: Tw(n) = Tw(1)+(n-1)c
Since Tw(1) & c are constants:
Tw(n) = c1+c2 n
hence:
Tw(n) = Θ(n)
Therefore:
worst case = O(n)
2. Average case calculation:
Tavg(n) = Tavg(n-1)+c
Tavg(n) = Tavg(n-2)+2c
Tavg(n) = Tavg(1)+(n-1)c
Tavg(n) = O(n)

Result:

The factorial was successfully implemented using iterative and recursive methods. Both methods have O(n) time complexity. The iterative method uses O (1) space, while the recursive method uses O(n) space.